BGP 101: Lab Guide

Carlos M. Martinez

2026-10-18

Version v0.1.0

Carlos M. Martinez
XT6 Labs

Version history

Version	Date

Table of contents

About this course **5**
References & Further Reading . . . 5

1 Lab 1 - Establishing Basic BGP Connectivity **7**
1.1 Objectives . . . 7
1.2 Topology . . . 7
1.3 Deployment . . . 7
1.4 Step 1: Connectivity Check . . . 7
1.5 Step 2: BGP Configuration . . . 8
1.6 Step 3: Verifying the Session . . . 8
1.7 Step 4: Inspecting Attributes . . . 8
1.8 Summary . . . 9

2 Lab 2 - iBGP, MP-BGP, and Route Origination **10**
2.1 Objectives . . . 10
2.2 Topology & Addressing . . . 10
2.3 Deployment . . . 10
2.4 Task 0: Interface and Loopback Configuration . . . 10
2.5 Task 1: The Internal Foundation (IGP) . . . 11
2.6 Task 2: Establishing iBGP over Loopbacks . . . 12
2.7 Task 3: Originating Prefixes (The Anchor) . . . 12
2.8 Task 4: Solving the Next-Hop Issue . . . 13
2.9 Task 5: Controlled Redistribution . . . 14
2.10 Task 6: Soft Resets . . . 15
2.11 Appendix: Containerlab Topology . . . 15

3 Lab 3 - Multi-Homing, Policy, and Traffic Engineering **17**
3.1 Objectives . . . 17
3.2 Topology & Addressing . . . 17
3.3 Deployment . . . 18
3.4 Task 1: Basic Peering & Route Origination . . . 18
3.5 Task 2: Internal Propagation (iBGP) . . . 18
3.6 Task 3: Inbound Filtering (The “Default Only” Policy) . . . 19
3.7 Task 4: Traffic Engineering (Local Preference) . . . 19
3.8 Bonus: Failover Test . . . 19

4 Lab 4 - Capstone: The Regional Service Provider **20**
4.1 Objectives . . . 20
4.2 Topology . . . 20
4.3 Deployment . . . 20
4.4 Task 1: The Foundation (IGP & iBGP) . . . 21

4.5 Task 2: Customer Onboarding (AS 65100) 21
4.6 Task 3: Multi-Provider Transit (TE) 21
4.7 Task 4: Advanced Policy (BGP Communities) 22
4.8 Final Verification Checklist 22

About this course

This course provides a comprehensive introduction to the **Border Gateway Protocol (BGP)**, the "glue" that holds the Internet together. It explores how network operators exchange routing information, enforce policies, and exert control over traffic flows to maintain a stable, loop-free global network.

Unlike theoretical deep dives, this course is designed for **practical observability and hands-on engineering**. We move beyond the "how it works" to the "how to build and fix it." By using modern, containerized environments like **FRR (Free Range Routing)** and **Containerlab**, students can simulate complex, multi-AS topologies on their own machines, gaining the confidence to manage real-world infrastructure.

Key topics covered include:

- **Core Concepts:** Understanding Autonomous Systems (ASNs), the BGP Finite State Machine (FSM), and the fundamental distinction between Internal (iBGP) and External (eBGP) sessions.
- **Protocol Mechanics & Messaging:** A deep dive into BGP message types and the underlying TCP-based transport that ensures reliability.
- **Path Selection & Policy:** Mastering the BGP Best Path Selection algorithm and understanding how Path Attributes (Weight, Local Pref, MED, AS_PATH) are used to manipulate traffic flow.
- **Internal BGP (iBGP):** Navigating the complexities of internal route propagation, the split-horizon rule, and scalable architectures using Route Reflectors.
- **Route Origination & Hygiene:** Professional techniques for announcing prefixes to the world, including the "Exact Match" rule, route anchoring with Null0/blackhole routes, and safe redistribution strategies.
- **Verification & Troubleshooting:** Developing a systematic approach to debugging BGP sessions, including interpreting the BGP table, verifying advertisements, and using soft-resets to apply policy changes without disruption.

References & Further Reading

Official Specifications

- **RFC 4271:** A Border Gateway Protocol 4 (BGP-4).
- **RFC 4456:** BGP Route Reflection: An Alternative to Full Mesh Internal BGP.
- **RFC 1997:** BGP Communities Attribute.

Tools used in this course

- **FRR (Free Range Routing):** The IP routing protocol suite for Linux and Unix platforms.
- **Containerlab:** Orchestration for container-based networking labs.

Community Resources

- **Hurricane Electric BGP Toolkit:** Essential tool for checking global routing information and ASN details.
- **NANOG BGP Tutorials:** High-quality presentations and videos from the North American Network Operators Group.
- **BGPStream:** A free resource for monitoring BGP hijacking, leaks, and outages.

1 Lab 1 - Establishing Basic BGP Connectivity

1.1 Objectives

- Configure a basic eBGP session between two routers.
- Verify the BGP Finite State Machine (FSM) transitions.
- Originate and verify a prefix announcement.
- Inspect BGP Path Attributes (AS_PATH, NEXT_HOP).

1.2 Topology

We will use a simple point-to-point topology: - **Router A (AS 65001)** - Interface IP: 10.0.0.1/30 - Loopback: 1.1.1.1/32 - **Router B (AS 65002)** - Interface IP: 10.0.0.2/30 - Loopback: 2.2.2.2/32

1.3 Deployment

Use the provided `lab-1.clab.yml` to start the topology:

```
sudo containerlab deploy -t lab-1.clab.yml
```

1.4 Step 1: Connectivity Check

Before BGP can establish a session, the underlying IP connectivity must be functional.

```
Router-A# ping 10.0.0.2
Type escape sequence to abort.
Sending 5, 100-byte ICMP Echos to 10.0.0.2, timeout is 2 seconds:
!!!!!
Success rate is 100 percent (5/5)
```

1.5 Step 2: BGP Configuration

Configure the BGP process and the neighbor relationship.

1.5.1 Router A

```
router bgp 65001
 neighbor 10.0.0.2 remote-as 65002
 network 1.1.1.1 mask 255.255.255.255
```

1.5.2 Router B

```
router bgp 65002
 neighbor 10.0.0.1 remote-as 65001
 network 2.2.2.2 mask 255.255.255.255
```

1.6 Step 3: Verifying the Session

Observe the BGP state. It should move from `Active` or `OpenSent` to `Established`.

```
Router-A# show ip bgp summary
BGP router identifier 1.1.1.1, local AS number 65001
...
Neighbor        V           AS MsgRcvd MsgSent   TblVer  InQ OutQ Up/Down  State/PfxRcd
10.0.0.2        4        65002      10      10        3    0    0 00:05:21        1
```

1.7 Step 4: Inspecting Attributes

Check how the prefixes appear in the BGP table and note the attributes discussed in Module 03.

```
Router-A# show ip bgp
BGP table version is 3, local router ID is 1.1.1.1
Status codes: s suppressed, d damped, h history, * valid, > best, i - internal,
              r RIB-failure, S Stale, m multipath, b backup-path, f RT-Filter,
              x best-external, a additional-path, c RIB-compressed,
Origin codes: i - IGP, e - EGP, ? - incomplete

   Network          Next Hop            Metric LocPrf Weight Path
*> 1.1.1.1/32       0.0.0.0                  0         32768 i
*> 2.2.2.2/32       10.0.0.2                 0             0 65002 i
```

Observations: - **AS_PATH:** Notice the `Path` attribute for `2.2.2.2/32` shows `65002`. - **NEXT_HOP:** The `Next Hop` for the remote prefix is the interface IP of Router B. - **ORIGIN:** The `i` at the end indicates the route was originated via an IGP (the `network` command).

1.8 Summary

In this lab, you successfully established an eBGP session and exchanged routes. You observed the fundamental attributes that BGP uses to track the origin and path of a prefix.

2 Lab 2 - iBGP, MP-BGP, and Route Origination

2.1 Objectives

- Establish an iBGP session using Loopback interfaces.
- Originate IPv4 and IPv6 prefixes using the `network` command and "Anchoring".
- Observe and fix the iBGP Next-Hop issue using `next-hop-self`.
- Perform controlled redistribution from an IGP into BGP.
- Use soft-reset techniques to apply policy changes.

2.2 Topology & Addressing

We will use a 3-router topology in Containerlab:

Device	Interface	IPv4 Address	IPv6 Address	BGP Role
R1	Loopback0	1.1.1.1/32	2001:db8:1::1/128	AS 65000 Border
R1	eth1 (to R2)	10.0.12.1/24	2001:db8:12::1/64	-
R1	eth2 (to R3)	10.0.13.1/24	2001:db8:13::1/64	-
R2	Loopback0	2.2.2.2/32	2001:db8:2::2/128	AS 65000 Internal
R2	eth1 (to R1)	10.0.12.2/24	2001:db8:12::2/64	-
R3	eth1 (to R1)	10.0.13.3/24	2001:db8:13::3/64	AS 65001 Provider

---

2.3 Deployment

Use the provided `lab-2.clab.yml` to start the topology:

```
sudo containerlab deploy -t lab-2.clab.yml
```

2.4 Task 0: Interface and Loopback Configuration

Before starting the BGP configuration, you must define the Loopbacks. In FRR, these are treated like any other interface.

2.4.1 Configuration (R1)

```
interface lo
 ip address 1.1.1.1/32
 ipv6 address 2001:db8:1::1/128
!
interface eth1
 ip address 10.0.12.1/24
 ipv6 address 2001:db8:12::1/64
```

2.4.2 Configuration (R2)

```
interface lo
 ip address 2.2.2.2/32
 ipv6 address 2001:db8:2::2/128
!
interface eth1
 ip address 10.0.12.2/24
 ipv6 address 2001:db8:12::2/64
```

2.5 Task 1: The Internal Foundation (IGP)

Before BGP, our internal routers must know how to reach each other's Loopbacks. We will use static routes for simplicity in this lab.

2.5.1 Configuration (R1)

```
ip route 2.2.2.2/32 10.0.12.2
ipv6 route 2001:db8:2::2/128 2001:db8:12::2
```

2.5.2 Verification

Ensure you can ping R2's loopback from R1:

```
R1# ping 2.2.2.2
!!!!! (Success)
```

2.6 Task 2: Establishing iBGP over Loopbacks

Now we establish the BGP session. In FRR, we must specify the `update-source` to ensure the TCP session initiates from the Loopback IP.

2.6.1 Configuration (R1)

```
router bgp 65000
 neighbor 2.2.2.2 remote-as 65000
 neighbor 2.2.2.2 update-source 1.1.1.1
 !
 address-family ipv4 unicast
  neighbor 2.2.2.2 activate
 exit-address-family
```

2.6.2 Verification

Check the session state:

```
R1# show ip bgp summary
Neighbor        V         AS   MsgRcvd MsgSent   TblVer  InQ OutQ  Up/Down     State/PfxRcd
2.2.2.2         4      65000        10      10        0    0    0 00:05:12            0
```

Expected Result: State should be numerical (e.g., 0) or "Established", not "Active" or "Idle".

2.7 Task 3: Originating Prefixes (The Anchor)

We will announce `192.0.2.0/24` and `2001:db8:beef::/48`.

2.7.1 Configuration (R1)

```
ip route 192.0.2.0/24 blackhole
ipv6 route 2001:db8:beef::/48 blackhole
!
router bgp 65000
 address-family ipv4 unicast
  network 192.0.2.0/24
 exit-address-family
 !
 address-family ipv6 unicast
  network 2001:db8:beef::/48
 exit-address-family
```

2.7.2 Verification

Check if R1 is actually announcing these:

```
R1# show ip bgp self
Network          Next Hop            Metric LocPrf Weight Path
*> 192.0.2.0/24   0.0.0.0                  0         32768 i
```

Expected Result: The `*>` symbols indicate the route is Valid and Best.

2.8 Task 4: Solving the Next-Hop Issue

R1 has an eBGP session with R3 (Provider). R3 is sending a route `100.64.0.0/10` to R1.

2.8.1 Observation (on R2)

Check the route on the internal router:

```
R2# show ip bgp
Network          Next Hop            Metric LocPrf Weight Path
* i 100.64.0.0/10   10.0.13.3               0    100      0 65001 i
```

Expected Result: Notice the `i` (learned via iBGP) but the lack of a `>`. The Next-Hop `10.0.13.3` is inaccessible to R2!

2.8.2 The Fix (on R1)

```
router bgp 65000
 address-family ipv4 unicast
  neighbor 2.2.2.2 next-hop-self
```

2.8.3 Verification (on R2)

```
R2# show ip bgp
Network          Next Hop            Metric LocPrf Weight Path
*>i 100.64.0.0/10   1.1.1.1                  0    100      0 65001 i
```

Expected Result: The Next-Hop changed to `1.1.1.1` (R1's Loopback), which R2 knows how to reach. The route is now `*>` (Best).

2.9 Task 5: Controlled Redistribution

We want to redistribute R1's connected interfaces but ONLY the one used for the "Lab-Services" (`172.16.1.0/24`).

2.9.1 Configuration (R1)

```
ip prefix-list PL-SERVICES permit 172.16.1.0/24
!
route-map RM-SERVICES permit 10
 match ip address prefix-list PL-SERVICES
!
router bgp 65000
 address-family ipv4 unicast
  redistribute connected route-map RM-SERVICES
```

2.9.2 Verification

Ensure other connected interfaces (like the one to R3) are NOT in the BGP table:

```
R1# show ip bgp
...
*> 172.16.1.0/24   0.0.0.0                  0         32768 ?
```

Expected Result: Only the permitted subnet appears. Other connected interfaces are filtered.

2.10 Task 6: Soft Resets

Modify the `RM-SERVICES` route-map to set a MED of 50, then apply the change without dropping the session.

2.10.1 Configuration (R1)

```
route-map RM-SERVICES permit 10
 set metric 50
!
! Apply change softly
clear ip bgp 2.2.2.2 soft out
```

2.10.2 Verification (on R2)

```
R2# show ip bgp 172.16.1.0
BGP routing table entry for 172.16.1.0/24, version 5
...
  Metric: 50, Localpref: 100, Weight: 0
```

Expected Result: The Metric (MED) should now be 50.

2.11 Appendix: Containerlab Topology

You can use the following `lab2.clab.yml` file to instantiate the lab environment. This topology uses the official FRR image and sets up the links according to the addressing table.

```yaml
name: bgp-lab2

topology:
  nodes:
    r1:
      kind: linux
      image: frrouting/frr:latest
    r2:
      kind: linux
      image: frrouting/frr:latest
    r3:
      kind: linux
      image: frrouting/frr:latest

  links:
    - endpoints: ["r1:eth1", "r2:eth1"]
    - endpoints: ["r1:eth2", "r3:eth1"]
```

Note: To run this lab, save the content above as `lab2.clab.yml` and run `sudo containerlab deploy -t lab2.clab.yml`. Once deployed, you can access the routers using `docker exec -it clab-bgp-lab2-r1 vtysh`.

3 Lab 3 - Multi-Homing, Policy, and Traffic Engineering

3.1 Objectives

- Implement BGP in a dual-homed environment with two different ISPs.
- Originate prefixes with high availability using "Anchoring" on multiple border routers.
- Solve iBGP propagation issues in a non-full-mesh environment.
- Implement inbound filtering to control routing table size.
- Use **Local Preference** to steer outbound traffic toward a primary ISP.

3.2 Topology & Addressing

This lab uses four routers: two in your AS (AS 65000) and two representing external ISPs.

```
graph LR
    subgraph AS 65001 [ISP-1]
        ISP1[ISP-1]
    end

    subgraph AS 65000 [My Network]
        R1[Border-A] --- R2[Core]
        R2 --- R4[Border-B]
    end

    subgraph AS 65002 [ISP-2]
        ISP2[ISP-2]
    end

    ISP1 --- R1
    R4 --- ISP2
```

Device	Interface	IP Address	BGP Role
R1	eth1 (to ISP-1)	10.0.1.1/24	Border-A (AS 65000)
R1	eth2 (to R2)	10.0.12.1/24	Border-A (AS 65000)
R2	eth1 (to R1)	10.0.12.2/24	Core (AS 65000)
R2	eth2 (to R4)	10.0.24.2/24	Core (AS 65000)
R4	eth1 (to R2)	10.0.24.4/24	Border-B (AS 65000)

Device	Interface	IP Address	BGP Role
R4	eth2 (to ISP-2)	10.0.2.4/24	Border-B (AS 65000)

3.3 Deployment

Use the provided `lab-3.clab.yml` to start the topology:

```
sudo containerlab deploy -t lab-3.clab.yml
```

3.4 Task 1: Basic Peering & Route Origination

Your task is to announce your network `192.0.2.0/24` to the world.

1. **Anchoring:** On both R1 and R4, create a blackhole route for your network.
   ```
   ip route 192.0.2.0/24 blackhole
   ```
2. **BGP Config:** Enable BGP on R1 and R4, establish eBGP with your respective ISPs, and use the `network` command.
3. **Verification:** Check ISP-1 and ISP-2. Do they see `192.0.2.0/24` in their BGP tables?

3.5 Task 2: Internal Propagation (iBGP)

Now, establish iBGP sessions between **R1-R2** and **R4-R2**.

1. **The Goal:** R2 should learn the full Internet (simulated) from both R1 and R4.
2. **The Problem:** Look at the BGP table on R2 (`show ip bgp`). Why are the routes learned from the ISPs marked as "unreachable" or "invalid"?
3. **The Fix:** Apply `next-hop-self` on R1 and R4 for their iBGP sessions toward R2.
   ```
   ! Example on R1
   router bgp 65000
    address-family ipv4 unicast
     neighbor 10.0.12.2 next-hop-self
    exit-address-family
   ```

3.6 Task 3: Inbound Filtering (The "Default Only" Policy)

ISP-1 is sending you many routes, but you only want a **Default Route** to save memory on R1.

1. **Define Policy:** Create a prefix-list that only permits `0.0.0.0/0`.
2. **Apply:** Use a route-map to apply this prefix-list **inbound** on R1's session with ISP-1.
3. **Verification:** Run `show ip bgp` on R1. You should only see `0.0.0.0/0` from ISP-1.

3.7 Task 4: Traffic Engineering (Local Preference)

Currently, R2 will pick a path based on its internal proximity. We want to force R2 to **always** use R1 (ISP-1) as the exit point for all Internet traffic.

1. **Identify the Tool:** We will use **Local Preference**.
2. **Configuration (R1):** Create a route-map that sets the local-pref to 200 and apply it inbound from ISP-1.

```
route-map RM-PRIMARY-ISP permit 10
 set local-preference 200
!
router bgp 65000
 address-family ipv4 unicast
  neighbor 10.0.1.100 route-map RM-PRIMARY-ISP in
```

3. **Soft Reset:** Don't forget to run `clear ip bgp 10.0.1.100 soft in`.
4. **Verification:**
   - On R2, run `show ip bgp`. Is the path via R1 (10.0.12.1) now preferred (marked with `>`)?
   - Perform a `traceroute` from R2 to an external IP. Does it exit via R1?

3.8 Bonus: Failover Test

1. Shut down the interface on R1 toward ISP-1.
2. Watch how R2 automatically updates its BGP table to use the path via R4 (ISP-2).
3. Bring the interface back up and watch the traffic return to R1 once the Local-Pref 200 routes are re-learned.

4 Lab 4 - Capstone: The Regional Service Provider

4.1 Objectives

- Build a scalable ISP core using **OSPF** and **iBGP Route Reflection**.
- Implement secure customer onboarding with prefix filtering and limits.
- Manage multi-provider transit using **Local Preference** and **AS-PATH Prepending**.
- Implement advanced Traffic Engineering using **BGP Communities** for Remote Triggered Blackholing (RTBH).

4.2 Topology

You are operating **AS 65000**. Your network consists of a central core router and two border routers.

```
graph TD
    subgraph AS65000 [My ISP - AS 65000]
        C1[CORE-1 / RR] --- BW[BR-WEST]
        C1 --- BE[BR-EAST]
    end

    UP1[UP-1 / AS 65001] --- BW
    UP2[UP-2 / AS 65002] --- BE
    CUST[CUST-A / AS 65100] --- C1
```

4.3 Deployment

Use the provided `lab-4.clab.yml` to start the topology:

```
sudo containerlab deploy -t lab-4.clab.yml
```

4.4 Task 1: The Foundation (IGP & iBGP)

Before BGP can work, your internal routers must be able to reach each other's Loopbacks.

1. **Loopbacks:** Assign IP addresses to `lo` on CORE-1 (1.1.1.1/32), BR-WEST (2.2.2.2/32), and BR-EAST (3.3.3.3/32).
2. **OSPF:** Enable OSPF on all internal links and loopbacks within AS 65000.
3. **iBGP Route Reflection:**
   - Configure CORE-1 as a **Route Reflector**.
   - BR-WEST and BR-EAST should be neighbors of CORE-1 (using loopbacks).
   - On CORE-1, set the border routers as `route-reflector-client`.

4.5 Task 2: Customer Onboarding (AS 65100)

CUST-A is a new customer. You have assigned them the block `203.0.113.0/24`.

1. **Strict Filtering:** Create a prefix-list on CORE-1 that *only* allows `203.0.113.0/24` from CUST-A.
2. **Prefix Limit:** Implement `maximum-prefix 5` on the neighbor session to prevent them from sending too many routes.
3. **Verification:** Ensure CORE-1 sees the route and reflects it to the border routers.

4.6 Task 3: Multi-Provider Transit (TE)

You have two upstreams. UP-1 (AS 65001) is your primary provider.

1. **Outbound TE (Local-Pref):** On BR-WEST, set a **Local Preference of 200** for all routes received from UP-1.
2. **Inbound TE (AS-Path Prepending):** On BR-EAST, make your network look less attractive to the world. Prepend your ASN three times (`65000 65000 65000`) for all outbound advertisements to UP-2.
3. **Verification:**
   - Check CORE-1: Does it prefer the path through BR-WEST for external routes?
   - Check UP-2: Does it see your prefixes with a long AS_PATH?

4.7 Task 4: Advanced Policy (BGP Communities)

CUST-A wants a way to "Blackhole" specific IPs during a DDoS attack.

1. **Define the Community:** Create a policy where if a route is received with the community `65000:666`, the next-hop is set to `blackhole`.
2. **Implementation:** Apply this policy on CORE-1 for the CUST-A session.
3. **Trigger:** On CUST-A, announce `203.0.113.50/32` with the community `65000:666`.
4. **Verification:** Check the routing table on BR-WEST. Traffic for `203.0.113.50` should now be discarded.

4.8 Final Verification Checklist

- ☐ CORE-1 has iBGP sessions up with both Border routers.
- ☐ OSPF is providing reachability between loopbacks.
- ☐ CUST-A can reach the simulated internet (UP-1/UP-2).
- ☐ Outbound traffic from the ISP prefers UP-1.
- ☐ The Blackhole community correctly drops traffic for the targeted IP.